

Druge vsebine

dr. Matej Šprogar

Model View Controller

MVC je arhitekturni vzorec za organizacijo kode v spletnih in programskih aplikacijah. Ločuje **podatke**, **uporabniški vmesnik** in **logiko**, da je razvoj bolj strukturiran, berljiv in vzdržljiv.

Sestavni deli:

- **Model** – predstavlja **poslovno logiko** in dela s podatki
- **View** – predstavlja **uporabniški vmesnik** (kaj uporabnik vidi)
- **Controller** – **posreduje** med modelom in pogledom; upravlja dogodke, usmerja podatke

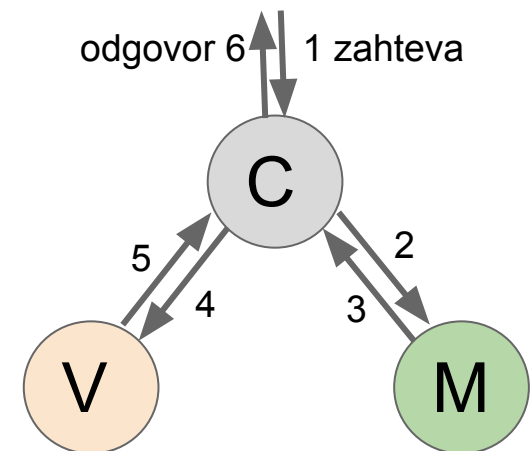
Cilj:

- Ločevanje skrbi (Separation of Concerns)
- Enostavnejše testiranje, razširljivost in timsko delo

Kako deluje MVC?

Potek komunikacije:

1. **Uporabnik** sproži zahtevo (npr. klik gumba), ki *pride* do **Krmilnika**
2. **Krmilnik** po prejemu zahteve pokliče ustrezno logiko v **Modelu**
3. **Model** obdela podatke in vrne rezultat
4. **Krmilnik** posreduje prejete podatke **Pogledu**
5. **Pogled** vrne **Krmilniku** obliko primerno za uporabnika
6. **Krmilnik** pošlje odgovor uporabniku



MVC

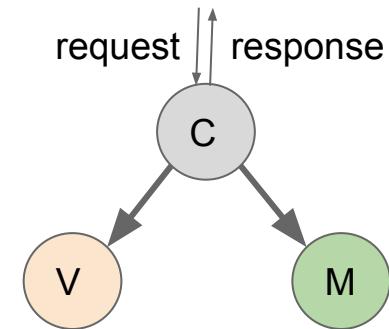
Prednosti:

- ◆ Ločena odgovornost = bolj organizirana koda
- ◆ Lažje testiranje in vzdrževanje
- ◆ Primeren za **spletne, namizne in mobilne aplikacije**
- ◆ Povečana možnost ponovne uporabe kode

Slabosti:

- ◆ Za preproste aplikacije lahko prekompleksen
- ◆ Več začetnega učenja za nove razvijalce
- ◆ Zahteva dobro načrtovanje in disciplino pri organizaciji

MVC - povzetek



Če govori REST o zunanji komunikaciji, govori **MVC** o zasnovi aplikacije.

Ideja je v **razdelitvi odgovornosti** in **razslojitvi** kode. Razmejuje odgovornosti (*SeparationOfConcerns*), kar omogoči lažje spremembe, ponovno uporabo kode in nadgradnje.

Kontroler zgolj odloča, kateri model bo opravil delo in kateri view bo prikazal modelove rezultate. Model ne ve, za koga in zakaj sploh dela kar dela, view pa ne ve, od kod so prišli podatki/rezultati in za koga jih mora oblikovati...

p.s. Nekatera ogrodja predvidevajo sicer tudi povezavo $M \rightarrow V$.

Frontend pristop

React:

frontend (UI) teče v browserju, backend daje JSON

Browser generira (ali *hidrira*) UI (komponente)

Backend = REST API

Browser → Server(→ JSON) → Browser(React render)

Frontend z Reactom

Razbijanje kompleksnosti (component-based)

Velika stran → razdeljena na komponente (Header, Table, Form)

Vsaka komponenta rešuje manjši problem

Komponente se sestavijo v drevo (app)

Build & deployment:

Koda (JSX) → build (Vite/Webpack) → bundle (.js)

Bundle nastane ob deploymentu (npm run build)

Browser potem samo naloži in izvede bundle

Prednosti in slabosti Reacta



Prednosti:

- ◆ Razbijanje kompleksnosti problema (komponente)
- ◆ Dinamičen in odziven UI
- ◆ Razbremenjen strežnik
- ◆ Build optimizacija



Slabosti:

- ◆ Večja kompleksnost same tehnologije
- ◆ SEO in začetni load (rešujemo s SSR + "hidracijo")
- ◆ Frontend ni skrit
- ◆ Overkill za enostavne aplikacije
- ◆ Hitro spreminjanje ekosistema

MVC | frontend

MVC (server-side)

Controller + Model + View → vse na serverju

- backend:
 - logika
 - podatki
 - render HTML
- frontend:
 - skoraj nič (samo prikaz)

👉 kompleksnost je **centralizirana na backendu**

React + REST

Frontend (React) + Backend (API)

- backend:
 - poslovna logika
 - validacija
 - podatki
- frontend:
 - UI logika
 - stanje (state)
 - routing
 - interakcije

👉 kompleksnost se **razdeli na dva dela.**

"zastarelo", "slabo", "odpisano"?

Ker strani osnovane na predlogah (PHP, JSP, ASP...) omogočajo scripting znotraj HTML, so programerji dejansko prevečkrat pisali *celotno* kodo kar v njih, brez uporabe slojev, MVC... Posledično je bila koda nepregledna in nemogoča za vzdrževanje, saj je združila *uporabniški vmesnik s poslovno logiko* in s *podatkovnim slojem*.

Težava torej ni toliko v tehnologiji, kot v njeni slabi uporabi.

Mlajše *anotirane* šablone ali *komponente* spet prinašajo svoje probleme (kompleksno, nepregledno, krhko, ...).

Spreminjajo se ogrodja in jeziki, čas dokaže kvaliteto!

HTML (Web) API

Web API-ji so **vneprej pripravljene funkcionalnosti**, ki jih brskalniki ponujajo spletnim razvijalcem za **interakcijo z brskalnikom ali napravo**. Na voljo so **neposredno znotraj brskalnika**, brez dodatnih knjižnic. Standardizirani s strani W3C/WHATWG,

Pomembnejši Web API-ji:

DOM API (dostop in manipulacija HTML dokumenta)

Web Storage API (trajen localStorage in začasen sessionStorage na strani odjemalca)

Fetch API (pošiljanje asinhronih HTTP zahtev)

Geolocation API (Pridobi uporabnikovo geografsko lokacijo)

Canvas API (Omogoča risanje 2D grafike, animacij in vizualizacij neposredno v <canvas> elementu)

...

Optimizacija za iskalnike - SEO

Search Engine Optimisation je proces izboljševanja spletne strani, da se višje uvrsti v iskalnikih (npr. Google) in privabi več *organskih* (neplačanih) obiskovalcev.



Cilj:

Povečati vidnost spletne strani v rezultatih iskanja za relevantne ključne besede.



Ključni elementi:

- **On-page SEO** – vsebina, naslovi, meta oznake, struktura strani
- **Off-page SEO** – povezave (backlinki), družbena omrežja, ugled
- **Tehnični SEO** – hitrost strani, mobilna odzivnost, indeksacija, sitemap



Zakaj je pomemben?

- 75 % uporabnikov ne klikne dalje od **1. strani** Googla
- Več organskega prometa = več priložnosti za prodajo, prijave, konverzije

Najboljše SEO prakse

✓ On-page SEO:

- Uporabi **ključne besede** naravno v naslovih, podnaslovih in vsebini
- Optimiziraj **meta opise in naslove strani**
- Uporabi **alt besedilo** za slike
- Struktura: ustrezna uporaba značk

✓ Tehnični SEO:

- Stran naj bo **hitro dostopna** (minimiziraj CSS/JS, uporabi CDN)
- **Prilagojena mobilnim napravam** (responsive dizajn)
- Uporabi **sitemap.xml** in **robots.txt**
- Omogoči **HTTPS** (SSL certifikat)

✓ Off-page SEO:

- Gradnja kakovostnih **povratnih povezav** (backlinkov)
- Aktivnost na družbenih omrežjih
- Objavljanje gostujočih člankov (guest posts)

! Napake, ki se jih izogibaj:

- Prekomerna uporaba ključnih besed ("keyword stuffing")
- Duplicirana vsebina
- Slaba struktura URL-jev (npr. example.com/page?id=123)
- Manjkajoče meta oznake

SEO pojmi in metrike

♦ Impressions (prikazi)

Kaj je: Kolikokrat je bila tvoja stran prikazana v rezultatih iskanja (ne nujno kliknjena).

Primer: 1200 prikazov za nek iskalni izraz.

♦ CTR – Click-Through Rate (stopnja klikov)

Kaj je: Delež uporabnikov, ki kliknejo povezavo izmed vseh, ki so jo videli.

Formula: $(\text{Kliki} / \text{Prikazi}) \times 100$

Zakaj je pomembno: Meri, kako privlačna je tvoja vsebina v iskalnikih.

♦ Organic Traffic (organski promet)

Kaj je: Obiskovalci, ki pridejo na stran prek *neplačanih rezultatov* iskanja.

Zakaj je pomemben: Glavni cilj SEO – pridobiti obisk brez oglaševanja.

♦ Conversion (konverzija)

Kaj je: število ali delež obiskovalcev, ki na tvojo spletno stran pridejo prek organskega iskanja (npr. Google) in nato naredijo neko želeno dejanje.

Formula: $\text{dejanja} / \text{organski_obiskovalci}$

♦ Bounce Rate (stopnja zapustitve)

Kaj je: Delež uporabnikov, ki zapustijo stran, ne da bi pogledali še kaj naokoli.

Pomen: Visok bounce rate lahko nakazuje slabo vsebino, počasno stran ali napačno ujemanje iskalnega namena.

♦ Time on Page (čas na strani)

Kaj je: Povprečni čas, ki ga uporabnik preživi na strani.

Zakaj je pomemben: Višji čas pogosto pomeni bolj kakovostno in uporabno vsebino.

3rd party cookies: Od FLoC do Topics API

Kaj je bil FLoC?

- Federated Learning of Cohorts
- Razvrščal uporabnike v skupine (cohorts) na podlagi zgodovine brskanja
- Namen: ciljanje oglasov brez 3rd-party piškotkov
- Težave:
 - Kritike glede zasebnosti (sledenje, diskriminacija)
 - Kompleksnost in pomanjkanje podpore drugih brskalnikov
 - Ukinjen leta 2022



Kaj je Google Topics API?

- Naslednik FLoC, del pobude *Privacy Sandbox*
- Brskalnik sledi interesom uporabnika (temam), ne obiskanim stranem
- Teme se delijo z oglaševalci, npr.: "Books", "Fitness", "Travel"
- Lokalna obdelava – brez pošiljanja zgodovine ali identitete

Topics API - zakaj in kako deluje

Kako deluje Topics API?

1. Uporabnik brska po spletu
2. Brskalnik shrani do 3 interesne teme na teden
3. Pri oglasni zahtevi brskalnik pošlje teme spletnemu mestu
4. Teme se menjajo vsake 3 tedne, brez sledenja posamezniku

Zakaj je boljši od FLoC?

- Večja zasebnost: brez ID-jev, brez skupin
- Uporabniški nadzor: možno je videti in odstraniti teme
- Enostavnost: teme so jasne in razumljive
- Manj tveganj: ni profiliranja preko več spletnih mest

Ključni cilj

**Ciljano oglaševanje brez sledenja uporabniku kot posamezniku.
Ravnotežje med zasebnostjo in učinkovitostjo oglaševanja.**

Nevarnosti uporabe sej

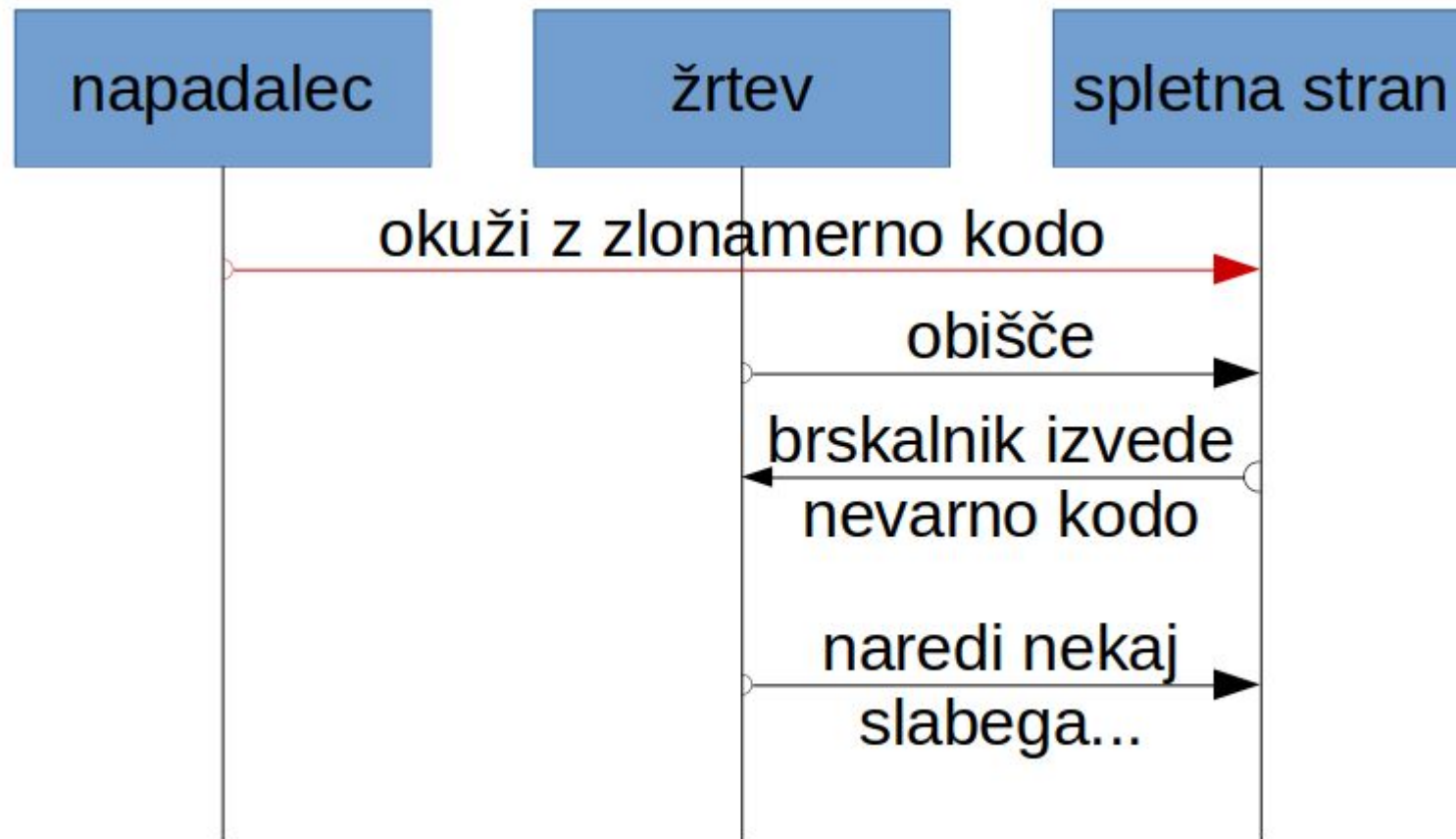
Piškoti s sejno oznako so varnostno kritični v smislu kraje oz. dostopa do vsebine piškota. Dostop do piškotov je mogoč ali zaradi varnostnih napak v brskalniku ali drugih napadov.

Obstaja mnogo načinov za pridobivanje sejnega ID:

1. Cross-Site Scripting (XSS) - Mogoče v slabi aplikaciji
2. Cross-Site Request Forgery (CSRF) - Mogoče v slabi aplikaciji
3. Man-in-the-Middle (*MitM*) - Nemogoče pri HTTPS in *Secure*
4. Podtikanje (*Fixation*) - Uporabniku podtaknemo svoj ID
5. Ugrabitev (*Hijacking*) - Uporabniku ukrademo njegov ID iz brskalnika
6. ...

Poseben predmet na temo varnost v tretjem letniku!

Cross Site Scripting (XSS)



XSS

Napadalec lahko na strežnik pošlje kodo preprosto tako, da jo v okviru podatkov obda z oznako `<script>`. Zato **NIKOLI ne prikaži nezavarovanega uporabniškega besedila**. Zavarovanje je možno ob sprejemu ali ob prikazu. Uporabimo lahko:

- a) *sanitizacijo* - izločitve “nevarnih” značk, ("`<script>...</script>`" → "")
in/ali
- b) *encoding* - (pre)kodiranje uporabniške vsebine! ("`<script>`" → "`<script>`")

Slaba praksa

`oglas.naslov`

`<%= $_REQUEST["abc"] %>`

`<%= ime %>`

Dobra praksa

→ `DOMPurify.sanitize(oglas.naslov)`

→ `<%= urlencode($_REQUEST["abc"]) %>`

→ `<%= encodeURIComponent(ime) %>`

Cross Site Request Forgery (CSRF)



CSRF

Ob obisku napadalčeve strani nehote sprožimo zahtevo na strežnik, ki ji naš brskalnik normalno pridoda piškote. Napadalec torej ne vidi sejnih podatkov, lahko pa *proži zahteve v našem imenu*.

Zaščita:

1. CSRF žeton (potrebna skrivnost, ki je napadalec ne more uganiti)
2. SameSite piškot nastavitev (*Strict* ali vsaj *Lax*)
3. Če že *SameSite Lax*, potem POST namesto GET za kritične operacije (idempotenten GET naj ne bi imel škodljivih posledic, POST pa v *Lax* ni dovoljen)

Wasm

Včasih JavaScript ni dovolj, zahteva preveč časa za nalaganje, parsanje in prevajanje oziroma je prepočasen (3D igre, virtualna resničnost, računalniški vid...). Takrat potrebujemo WebAssembly (*Wasm*).

WebAssembly je dejansko univerzalni izvajalni format, ki ga podpira brskalnik enako, kot podpira JS.

Originalno je bil mišljen kot način za uporabo C in C++ kode, obstajajo pa prevajalniki tudi za večino drugih jezikov.

Wasm ni nadomestilo za JS, ampak ga dopolnjuje. Wasm recimo nima direktnega dostopa do DOM, ampak preko JS.